AI for Economic Research

Lec. 9: Agentic AI, Research Workflows, and Claude Code

Zhigang Feng

Spring 2026

Lecture Agenda

1. Why economists should care
2. Four research patterns
3. What web chat can and cannot do
4. What makes a workflow agentic
5. Early end-to-end demo
6. Practical setup: dated tool notes
7. Workflow design and validation
8. Which tool fits which task
9. How a project stays organized
10. Mistakes, limits, and security
11. The context dilemma
12. Lecture 10 preview + wrap-up

Teaching claim: agentic AI is most useful when economists turn research tasks into explicit workflows with checks, files, and human judgment.

Framing

From chat tools to durable research workflows

Why Economists Should Care

The bottleneck in applied research is often not ideas. It is execution.

- Turning a good question into clean code, diagnostics, tables, and figures is slow
- Many research tasks are repetitive: reshape data, rerun specifications, trace sources, refactor scripts, update plots
- Agentic tools compress the time from *idea* to *first defensible result*
- That raises the return to the skills economists uniquely provide: question design, institutional knowledge, validation, and judgment

The promise is not "AI does research for you." The promise is faster implementation and faster iteration around a question you still have to define and defend.

Four Research Patterns

Pattern	Representative task	Main risk if you use the wrong workflow
Reusable skill	Referee-style paper review	Generic comments, hallucinated quotes, missing quality gate
Empirical pipeline	MEPS harmonization and figures	Wrong crosswalk, dropped survey weights, silent validation failure
Public-web dataset	Fellows metadata with provenance (field-level source trace)	Weak sources, hidden missingness, no audit trail
Full-lifecycle collaborator	Research memo → pipeline → critique → revision	Polished output built on weak framing or claims discipline

This lecture uses Claude Code as the concrete terminal-agent example, but the workflow ideas are meant for economists across fields, not only for computational macro.

What Remains Human Across the Four Patterns

You still own

- the research question
- the success criteria
- the coding and source rules
- the validation target
- the final interpretation

The agent accelerates

- repetitive execution
- file handling and refactoring
- promptable decomposition
- reruns after fixes
- draft diagnostics and reports

Why this matters for economists: the gain is not that judgment disappears. The gain is that disciplined execution becomes cheaper.

What Modern Web Chat Already Does Well

As of April 2026, a strong web-chat product is already useful for economists.

- Brainstorming model setups, empirical strategies, review rubrics, and robustness checks
- Summarizing papers, referee reports, and lecture notes
- Inspecting uploaded files and explaining code snippets
- Sometimes browsing the web or running limited sandboxed tools
- Helping you draft a specification before you open your editor

So the weak baseline is not "a browser cannot do anything." The weak baseline is a *single chat session without a durable project harness*.

What Still Breaks Without a Project Harness

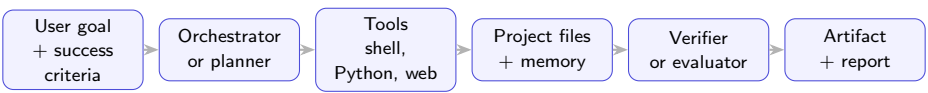
A single web-chat session often fails at the exact points where research projects become real.

1. **Durable context is weak.** Yesterday's conventions and corrections are not reliably embedded in today's work
2. **Reusable workflow is weak.** Your validation checklist lives in prompts, not in inspectable files
3. **Controlled execution is weak.** The model is not operating inside your actual project tree with your actual scripts and outputs
4. **Audit trail and provenance are weak.** It is hard to inspect which files, commands, checks, and sources produced the result

That is the real contrast. Agents are not magic because they are "smarter." They are useful because they can operate inside a structured project with files, tools, and repeatable workflows.

What Makes a Workflow Agentic Under the Hood?

An agentic workflow is not just text generation. It is a control loop wrapped around tools, files, and checks.



- **Orchestrator:** the loop deciding what to do next
- **Memory:** saved project state on disk, not just chat history
- **Verifier:** tests, benchmarks, or review checks deciding whether output survives

How Should Economists Evaluate an Agentic Workflow?

Criterion	Question to ask
Correctness	Did it compute, extract, or summarize the right object?
Reproducibility	Can a fresh run recreate the same artifact from the saved files and rules?
Auditability	Can you inspect the commands, checks, and source trail that produced the result?
Privacy boundary	Which files stay local, and which contents may still traverse the network?
Latency / cost	Is the wall-clock gain worth the token, debugging, and supervision cost?

Why this matters for economists: an impressive demo is not enough. A research workflow has to be checkable and defensible.

Spectrum: Chat, IDE Assistant, Terminal Agent

	Web chat	**IDE assistant**	**Terminal agent**
Best at	thinking, drafting	editing local code	end-to-end execution
Context	conversation + uploads	open files	full project tree
Typical role	architect's whiteboard	coding copilot	workflow operator
Economic example	design the specification	clean one regression file	run the full pipeline

Concrete example today: Claude Code. **General lesson:** the same workflow logic also appears in Codex CLI, Gemini CLI, Aider, and similar terminal-agent tools.

The Bridge: How a Chatbox Becomes a Terminal Agent

One key technical change: modern LLMs (post-2023) support **structured tool calls**. Instead of writing "you should run `python solve.py`," the model emits a machine-readable request:

```
{ "tool": "Bash", "command": "python aiyagari_v0.py" }
```

A **harness**—software installed on your Mac—executes this and returns the real output. The model reads it and decides what to do next. This loop is what makes something "agentic."

	claude.ai (browser)	**Claude Code (terminal)**
LLM model	same (Opus 4.6)	
Harness	none (sandboxed)	local CLI on your Mac
Can run Python?	no	yes—*your* Python, *your* machine
Sees real errors?	no	yes—loops back automatically

Same brain, different arms. The terminal agent is not a magically smarter model. The harness is what flips the switch from text-only output to text + actions inside a project.

Early Demo: From Prompt to Research Artifact

Running example: a MEPS pilot asking: *"Build a 2012–2014 uninsurance panel by age group with survey weights."*

1. **Prompt:** define the question, weights, and desired output
2. **Plan:** list files to download, harmonize, and validate
3. **Run:** parse the raw files, map year-specific variables, compute weighted rates
4. **Validate:** compare 2013 to an external benchmark, inspect the weight variable, save the crosswalk
5. **Output:** one figure, one CSV, one short note saying what passed and what remains uncertain

Same loop, different artifact:

- paper review $\rightarrow$ structured critique
- fellows dataset $\rightarrow$ provenance-aware CSV
- Aiyagari model $\rightarrow$ benchmarked solver output

Reason for showing this early: the workflow should be concrete before we start naming the project objects that support it.

Practical Setup

Tool-specific operational notes for Claude Code on a Mac

Installing Claude Code on Your Mac (Dated, Part 1)

Three operational steps:

1. **Install Node.js** (required runtime)
   - Easiest: `brew install node`
   - Alternative: download from `nodejs.org` (v18 or later)
2. **Install Claude Code:**

```
npm install -g @anthropic-ai/claude-code
```

3. **Verify:** `claude -version`

Interpretation: this is operational setup, not the conceptual point of the lecture.

Installing Claude Code on Your Mac (Dated, Part 2)

Access and first launch:

1. **Subscribe** at `claude.ai`
2. **Typical tiers:**
   - Pro ($20/mo): adequate for most tasks
   - Max ($100/mo): stronger reasoning, higher limits
3. **First launch:** `cd your-project-folder` then `claude`

Mac + OneDrive note: always `cd` into your project folder before running `claude`. Running from a `/Volumes/...` path can trigger permission errors.

Dated note: subscription tiers, model names, and installation details change quickly. Recheck the official docs before teaching or running the lab.

Why Git, Not OneDrive or Dropbox

The risk without Git:

- Claude Code can edit dozens of files autonomously in one session
- OneDrive and Dropbox sync by *copying*—two simultaneous writes create corrupt merge files
- No undo: if Claude makes bad changes, there is no "revert" without Git
- No audit trail: you cannot see what changed or when

What Git gives you:

- `git diff` shows exactly what Claude changed before you accept it
- `git checkout -- .` reverts all uncommitted changes instantly—your emergency brake
- `git log` shows every change with a timestamp and message
- Works offline; cloud sync races with Claude's rapid file writes

Practical rule: store large data files on OneDrive. Store code and scripts in Git. Never use the same tool for both.

First Git Setup: Three Minutes

One-time global setup:

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com
    "
```

Per-project setup:

```
cd my-project
git init
echo "data/ output/ *.pdf" > .gitignore
git add .gitignore
git commit -m "initial setup"
```

The one rule:

- Commit *before* every Claude session
- Commit *after* validating results
- That is enough to start safely

Emergency brake:

```
git checkout -- .
```

Undoes *all* uncommitted file changes instantly. Your safety net when Claude goes wrong.

You do not need to know Git deeply. Three commands—`git add -A`, `git commit -m "..."`, `git checkout -- .`—are enough to work safely with Claude Code.

Under the Hood

Harness, memory, permissions, and the control loop

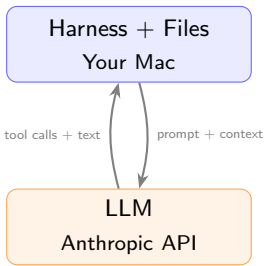
Three Components of a Terminal-Agent Setup

Claude Code has three physically distinct parts:

1. **The Harness** (lives on your Mac)
   CLI software you install. Reads your files, runs shell commands, manages the conversation loop.

2. **The LLM** (lives on Anthropic's servers)
   Every message is bundled with conversation history and sent via HTTPS to the API. The "thinking" happens in the cloud.

3. **Your Files** (live on your Mac)
   Python scripts, data CSVs, LaTeX—all on your disk. Code executes on your hardware with your packages.



Key: code can run locally and many project files stay on your machine.
That does *not* mean the whole workflow is automatically private.

Privacy Boundary: Local Execution Is Not the Same as Local Processing

- code may execute locally on your machine
- many project files may stay on disk locally
- if file contents, prompts, or tool outputs are sent back in the conversation, those contents may still traverse the network
- only a fully local or institutionally private deployment changes that boundary

Practical rule: local execution is not a license to paste restricted data into an agent workflow.

The Mac File Hierarchy: Global vs. Local

Two layers of configuration. **Local overrides global.**

Path	Scope	What lives here
`~/.claude/settings.json`	Global	Default permissions, model preference
`~/.claude/MEMORY.md`	Global	Cross-project memory (persists forever)
`~/.claude/skills/`	Global	Skills available in every project
`<project>/CLAUDE.md`	Local	Project brain, read every session
`<project>/.claude/settings.json`	Local	Project permissions (overrides global)
`<project>/.claude/skills/`	Local	This project's slash commands
`<project>/.claude/agents/`	Local	Specialized reviewer roles
`<project>/.claude/rules/`	Local	Always-on guardrails

On a Mac: `~` = `/Users/yourname`. To inspect: `ls ~/.claude/`

Rule: global `~/.claude/` is your personal defaults (apply everywhere). Local `.claude/` is the project's rules (apply only here).

Initialization: What Happens When You Type `claude`

Five steps run **before you type your first message**:

1. **Read global config** — loads `~/.claude/settings.json`: permission defaults, model preference
2. **Detect project context** — searches current directory (and parents) for `CLAUDE.md` and `.claude/`
3. **Load project brain** — reads `CLAUDE.md` into the context window
4. **Auto-load all rules** — every `.claude/rules/*.md` file is loaded silently
5. **Ready** — context window pre-populated before you say anything

Analogy: step 4 is a professor re-reading lecture notes before class. Students arrive; context is already loaded.

Cost warning: steps 3–4 consume tokens before you type anything. Keep `CLAUDE.md` and rules files concise.

The Permission System and Local Code Execution

Claude's toolbox:

Tool	What it does
`Bash`	Run any shell command
`Read`	Read a file from disk
`Write`	Create or overwrite a file
`Grep/Glob`	Search content / find files
`Agent`	Spawn a child Claude instance

The permission model:

- **Default:** Claude asks before every Bash call. You see the exact command first.
- **Whitelist:** add trusted commands to `.claude/settings.json`:

```
"allowedTools": [
  "Bash(python *)",
  "Bash(git *)"  ]
```

- **Bypass-permissions:** skips all prompts. Use only when you fully trust the task.

"Which Python ran this?" (common confusion):

- Claude's Bash inherits your shell's `$PATH`
- If you ran `conda activate myenv` before `claude`, *that* Python runs
- Add a `which python` check to your `CLAUDE.md` to confirm

Key Modes: Plan, Effort, and Session Control

Command / Key	What it does	When to use
`Shift+Tab`	**Plan mode**: Claude writes a plan, does NOT execute. Waits for approval.	Start every non-trivial task here
`/effort max`	Maximum reasoning depth: best quality, slower, costlier	Novel problems, economic correctness
`/effort default`	Balanced quality and speed	Routine work
`/cost`	Show tokens used + estimated USD for this session	Budget check before long tasks
`Esc`	Interrupt Claude mid-task immediately	When it's going the wrong way
`/compact`	Summarize conversation to free context window	After ~20 turns; between phases
`/clear`	Wipe context, start fresh	New unrelated task

The effort–cost tradeoff:

- `/effort max` + long context $\approx$ \$5–15 per complex session (Opus 4.6 pricing)
- Spend effort on *decisions*; save it on *execution*

Your First Session

The recommended workflow from scratch

Start Here: The Recommended Workflow for New Users

0. **Before Claude—pen + paper**
   Write one sentence: *"I want to compute X using method Y for question Z."*

1. **Enter plan mode (`Shift+Tab`)**
   Describe your goal. Claude drafts a plan. *Nothing runs yet.*

2. **Review and approve**
   Read the plan. Ask questions. Revise. Type "approved."

Why this matters: this is the cheapest place to catch a wrong task definition.

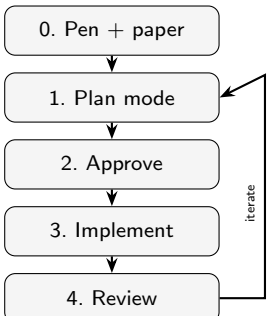
Common beginner mistake: skipping Step 0. Without a clear goal, Claude will confidently build the wrong thing.

Continue: Steps 3–5

3. **Implement**
   Claude executes the approved plan. Press `Esc` to stop if wrong.

4. **Review output**
   Does it make economic sense? Re-derive one number by hand.

5. **Iterate**
   Fix, extend one feature, compact context, and return to step 1.

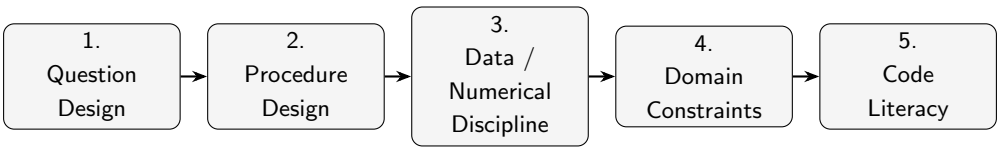


Practical takeaway: after each implementation pass, force one human check before you ask for the next extension.

The 5-Pillar Workflow

What stays human across models, empirical pipelines, and review tasks

The 5-Pillar Workflow



The throughline of the lecture:

- Pillars 1–4 determine what should be built and what counts as success
- Pillar 5 lets you verify what the agent actually built
- If the pillars are weak, agent output becomes faster nonsense

Pillar 1: Question Design

Before any workflow, decide what the economist is actually trying to learn.

- State the question in economist language: model object, empirical estimand, review brief, or dataset goal
- Decide what constitutes evidence: policy function, coefficient, figure, quote-level comment, or provenance-aware record
- Write down limiting cases before any code exists

Cross-domain point: if the question is vague, the workflow can still look polished while answering the wrong thing.

Pillar 2: Procedure Design

After the question is clear, decide what procedure is actually appropriate.

- Choose the right workflow: Bellman iteration, MEPS harmonization pipeline, paper-review decomposition, or batched metadata collection
- Decide what the acceptance criterion is: convergence, benchmark recovery, quote verification, or source completeness
- Convert the question into a bounded procedure the agent can implement and you can inspect

Cross-domain point: the agent can write VFI code, merge MEPS files, or draft a review report, but only you know which workflow is appropriate for the task.

Pillars 3 and 4: Data/Numerical Discipline and Domain Constraints

Pillar 3: Data / numerical discipline

- Grid design, interpolation, shock discretization, and stability checks for models
- Schema harmonization, weights, crosswalks, missingness, and saved source logs for data work
- Recognition of failure modes: kinks, dropped weights, broken mappings, or silently weak sourcing

Pillar 4: Domain constraints

- Advanced methods still matter: projection, perturbation, clustering decisions, source restrictions, claims discipline
- Knowing when the agent's convenient default is computationally, econometrically, or substantively wrong

Economics point: a workflow can run cleanly and still be wrong. Agents reduce coding cost; they do not remove approximation error, data fragility, or source discipline.

Pillar 5: Code Literacy and Division of Labor

You provide	Agent provides
Question, identification logic, and source rules	Fast code generation and refactoring
Workflow choice and benchmark design	Boilerplate implementation and debugging
Knowledge of the data, model, and institutions	File edits, script writing, and test runs
Economic interpretation and seminar defense	Iteration on visible errors and outputs

Research version of the slogan: the agent can be a strong RA, but you still have to be the principal investigator.

Validation Is Your Referee Checklist

Every iteration needs both workflow checks and economic checks.

Workflow checks

- Convergence achieved
- No NaN or Inf values
- Mappings, weights, or source logs saved where expected
- Diagnostics, residuals, or quote checks pass
- Script reproduces the same output twice

Economic checks

- Limiting cases recover known results
- Comparative statics, subgroup patterns, or review claims make sense
- Magnitudes and source coverage are plausible
- Inference uses the right standard errors, sample, and restrictions
- You can explain the result in plain economic language

If you cannot defend the output in a seminar, you are not done.

The Homotopy Workflow

Build from simple to complex, validate, then extend

The Homotopy Workflow

In workflow design, homotopy means solving an easier version first and then extending it toward the full research task.

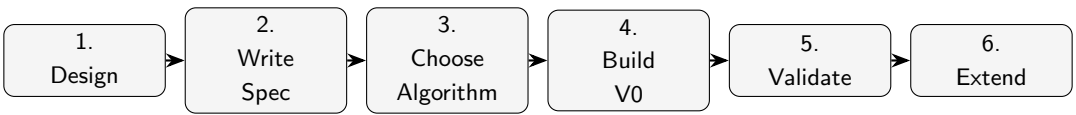
- Start from a version with clear intuition and known checks
- Use that validated baseline as the starting point for the next version
- Attribute new bugs to the new margin you just added
- Avoid asking the agent to solve the full complicated problem in one jump

Four examples of the same logic:

deterministic model $\rightarrow$ 3-year MEPS pilot $\rightarrow$ one-section review $\rightarrow$ seed roster only

Why this matters for agentic coding: the agent performs best when each step is bounded, testable, and attached to an existing working baseline.

Homotopy in Practice: Build from Simple to Complex



This replaces the fantasy prompt "solve my project."

- Start with a problem you can state and benchmark
- Give the agent bounded tasks with visible acceptance criteria
- Add one new ingredient at a time

Step 1: Design the Economic Problem

Before any code, write down the task in economist language.

- Research question in one sentence
- Objective and constraints, or the relevant source rules
- Equilibrium definition, estimand, or review standard if applicable
- Parameters, sample, source universe, and economic motivation
- Outputs you will treat as evidence

Aiyagari anchor:

$$\max \ \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_t) \quad \text{s.t.} \quad c_t + a_{t+1} = (1+r)a_t + wz_t, \ a_{t+1} \geq \underline{a}$$

Same principle elsewhere: a vague MEPS brief, a vague review request, or a vague fellows-data assignment also produces output that may be internally clean but substantively wrong.

Step 2: Write the Specification File

The specification file is the contract between you and the agent.

A good specification file states four things clearly:

1. assumptions and domain rules
2. data or source restrictions
3. procedure and validation target
4. required outputs and where to save them

Examples: CRRA utility and convergence tolerance for Aiyagari; weights and subgroup cuts for MEPS; quote-level evidence rules for paper review; official/institutional-source rules and provenance fields for fellows data.

Best practice: ask the agent to restate the specification in its own words before it writes code. That catches misunderstandings when they are still cheap.

Step 3: Choose the Procedure Explicitly

Pseudocode or a phase plan is where you catch design mistakes before implementation.

```
Phase A: ingest or define the object
Phase B: run the chosen procedure
Phase C: save diagnostics and checks
Phase D: report only what passed validation
```

Concrete procedures: VFI or EGM for Aiyagari, a 3-prompt pipeline for MEPS, section decomposition for paper review, and batched metadata collection for fellows data.

The point is not elegance. The point is that you and the agent agree on the procedure before code details begin to hide conceptual errors.

Step 4: Build V0 First

V0 is the simplest version with a benchmark you understand.

Aiyagari V0: deterministic income

- If $\beta(1+r) = 1$, savings policy should sit on the 45-degree line
- If $\beta(1+r) < 1$, the agent decumulates toward the borrowing constraint
- If $\beta(1+r) > 1$, assets grow

Discipline: do not ask the agent to add income risk, stationary distributions, and equilibrium search before V0 already passes the simple checks.

What "V0 First" Means Across Four Tasks

Task	V0	First validation target
Aiyagari model	deterministic household problem	recover simple savings logic and boundary behavior
MEPS pipeline	3-year pilot panel	weighted rates line up with an external benchmark
Paper review skill	one section or theorem block	quotes are real and comments are specific
Fellows dataset	seed roster only	source rules and provenance fields are actually recorded

Why economists should care: V0 keeps the first error interpretable. You want the first failure to be obvious, not hidden inside a giant workflow.

Steps 5 and 6: Validate, Then Extend One Margin at a Time

Version	New ingredient	Required benchmark
V0	Deterministic household	analytical cases
V1	One new model, data, or review margin	setting the new margin to zero recovers V0
V2	Saved diagnostics and state tracking	mass sums, benchmarks, quote checks, or source logs behave sensibly
V3	Full target workflow	final artifact survives validation and interpretation

Why this works with agents: when a new version breaks, both you and the agent know where to look.

Why This Workflow Works for Economists

- It keeps theory, computation, and validation aligned
- It gives the agent bounded tasks instead of open-ended wishes
- It preserves an audit trail of how the baseline was constructed
- It mirrors how good research assistants are actually trained: start with a simple task, verify it, then widen scope

Main lesson: the workflow is the real product. The particular agent you use can change.

Tool Choice

Which tool fits which research task

Choose the Tool That Matches the Task

Tool type	Use it for	Economic example
Web chat	framing and explanation	clarify identification logic, draft a model summary
IDE assistant	local code editing	vectorize one function or explain one traceback
Terminal agent	multi-step execution	restructure a project, run scripts, generate diagnostics, and iterate

As of April 2026: Claude Code, Codex CLI, Gemini CLI, and Aider are examples of terminal-agent workflows.

Why the Terminal Agent Often Becomes the Workhorse

- It can operate on the real project rather than on copied fragments
- It can run the code, inspect errors, and update multiple files
- It works well when the task is bounded but longer than one edit
- It still leaves the economist in the approval and validation loop

For empirical research, this is usually the sweet spot: enough autonomy to save time, not so much autonomy that you stop checking the economics.

Project Organization

How a terminal-agent project stays organized

Why Files on Disk Matter

The project harness turns today's chat into tomorrow's reusable workflow.

- Conventions live in files, not in your memory
- Validation steps become reproducible lab procedure
- New sessions can start from the project's actual state
- Coauthors and students can inspect what the workflow is supposed to be

Economist translation: the harness is like turning tacit RA practice into written lab protocol.

Why These Project Objects Exist: Coordination

Object	Research pain point it solves	Running example
`CLAUDE.md` / project memory	You do not want to re-explain the project every session	keep the MEPS weights rule or Aiyagari benchmark visible
Skill	A repeated workflow should not be improvised every time	paper-review pipeline with the same quality gates
Agent	Some review roles are easier to parallelize and specialize	code reviewer vs. domain reviewer

Reason for this slide: these objects are not product trivia. They are answers to recurring research coordination problems.

Why These Project Objects Exist: Control and Credibility

Object	Research pain point it solves	Running example
Rule	Some instructions should always be on	always save outputs to `output/` before claiming success
Verifier	Output should pass named checks before it counts	rerun script, inspect diagnostics, confirm files exist
Provenance log	Source discipline must survive beyond the chat	fellows dataset with field-level URLs and notes

Practical takeaway: workflow objects are worthwhile only if they reduce a real coordination or credibility failure.

Minimal Project Tree

```
my-research-project/
+-- CLAUDE.md
+-- .claude/
|   +-- skills/
|   |   +-- paper-review/
|   |       +-- SKILL.md
|   |       +-- references/
|   |           +-- referee-template.md
|   +-- agents/
|   |   +-- code-reviewer.md
|   |   +-- domain-reviewer.md
|   |   +-- verifier.md
|   +-- rules/
|   |   +-- iterative-workflow.md
|   +-- settings.json
+-- scripts/
+-- output/
+-- sources/
+-- quality_reports/
```

This is enough to explain the workflow without burying students in infrastructure. The exact research question can change; the organizational logic stays.

CLAUDE.md: The Team-Facing Project Memory

`CLAUDE.md` is the main shared instruction file for the project.

```
# CLAUDE.md
Project: MEPS uninsurance pilot for lecture 9
Language: Python
Goal: harmonize 2012--2014 files and produce one validated figure
Conventions: no hardcoded paths; save outputs to output/
Checks: use survey weights; save the year map; report what passed
```

Why students should care: it captures the kind of project context you would otherwise re-explain every session.

CLAUDE.md, Auto Memory, and Optional Course Logs

Three different persistence ideas should not be confused.

Mechanism	Who writes it	Why it exists
`CLAUDE.md`	you / team	durable project instructions and conventions
Auto memory	the tool	per-project learned patterns and corrections
Optional course log	you / course	explicit progress notes such as `MEMORY.md` or `progress.md`

Course recommendation: rely on `CLAUDE.md` for shared rules, treat auto memory as helpful but not authoritative, and keep explicit logs when you want students to inspect the evolution of a project.

Skills, Agents, and Rules: Three Different Jobs

Object	Economist analogy	What it does
Skill	saved RA protocol	reusable workflow such as solve, validate, review
Agent	specialist reviewer	a focused role: code quality, economics, verification
Rule	lab convention	always-on instruction such as plan first or save outputs here

This distinction matters because the student should know where to encode each idea.

Plain English vs. Encoded Skill

Plain English

Review this paper and tell me the top problems with exact quotes.

Use when

- Exploring
- Doing a one-off task
- Still deciding the workflow

Encoded skill

`/paper-review draft_A.pdf`

Use when

- The workflow should repeat exactly
- A student team needs one standard procedure
- Validation must be consistent across versions

Exploration starts in English. Reproducibility lives in skills.

Skill File Structure

```
.claude/skills/paper-review/
  SKILL.md
  references/
    referee-template.md
```

`SKILL.md` stores the repeatable protocol:

- what the command is called
- what steps it should follow
- what checks must pass before it reports success

Reference files store task-specific detail that would clutter the main instruction file.

Agents as Specialist Reviewers

Agent files are best understood as specialist review roles.

- `code-reviewer.md`: file hygiene, implementation quality, and reproducibility
- `domain-reviewer.md`: benchmark logic, identification, or economic consistency
- `verifier.md`: run the script, inspect diagnostics, check output or source logs exist

Economist analogy: one RA checks code style, another checks economics, and another reruns the table.

Subagents as Parallel Reviewers

Subagents matter because review tasks can happen in parallel.

- The main agent writes or edits code
- A code-reviewer and domain-reviewer can inspect the result at the same time
- The main conversation receives both reports and decides what to fix

Why students should care: the gain is not philosophical. It is wall-clock time. Focused review roles are faster and often clearer than one overloaded prompt.

Rules, Hooks, and Settings

Object	Loaded when	Typical use
Rules	during project work	always follow a planning or verification convention
Hooks	on tool events	run a formatter or log a command mechanically
Settings	at session start	permissions and project-level behavior

Practical distinction: if it is natural-language guidance, it usually belongs in a rule or skill. If it must happen mechanically every time, a hook is safer.

Tracing the Computational Example

Aiyagari as the computational anchor inside a broader workflow lecture

Aiyagari Recap: The Computational Anchor

Household problem

$$c_t + a_{t+1} = (1 + r)a_t + wz_t, \qquad a_{t+1} \geq \underline{a}$$

with idiosyncratic productivity z_t following a Markov chain.

Why it is a good lecture example

- it is the cleanest computational example in the course
- it admits a clean V0 $\rightarrow$ V1 $\rightarrow$ V2 progression
- it has obvious validation checkpoints that generalize to other workflows

Tracing the File Flow

```
1. Read CLAUDE.md
2. Read any relevant rules
3. Load /solve-model from SKILL.md
4. Read references/aiyagari-guide.md
5. Write plan or update quality_reports/
6. Edit scripts/aiyagari_v0.py
7. Run verifier and reviewers
8. Save outputs and logs
```

What students should notice: the workflow is file-driven. The agent is acting inside a structured research project, not improvising in a vacuum.

Tracing the Orchestrator Loop

```
You type: /solve-model aiyagari v1

Plan:
  read project instructions
  restate the target
  implement bounded changes

Review loop:
  run verifier
  run code reviewer
  run domain reviewer
  fix issues that matter
  report what changed and what passed
```

This is the core advantage of the harness: the review sequence is not a remembered prompt. It is part of the saved workflow.

Inside `CLAUDE.md` for the Aiyagari Project

```
# CLAUDE.md
Project: Aiyagari workflow for lecture 9
Goal: build v0, v1, and v2 sequentially
Outputs: policies, stationary distribution, diagnostics
Conventions: save figures to output/ and plans to quality_reports/
Validation: do not move to the next version until benchmarks pass
```

If the file is good, the project can survive a fresh session, a new RA, or a change in agent tool.

Inside `SKILL.md` and the Reference Guide

`SKILL.md`

```
name: solve-model
description: build and validate

Steps:
1. Read the model guide
2. Write or update a plan
3. Implement one version
4. Run required checks
5. Save outputs and report
```

`aiyagari-guide.md`

```
V0: deterministic benchmark
V1: add Markov income risk
V2: compute stationary dist.

Checks:
- zero-variance limit
- sensible mass at a_min
- stable diagnostics
```

Interpretation: the skill stores the protocol; the reference guide stores the model detail.

Mistakes, Limits, and Security

What can go wrong and how to stay disciplined

Mistake 1: Treating Git as Optional

- Agents are most useful when experimentation is reversible
- Git provides the audit trail for what changed and when
- A clean branch lets the agent try a refactor without destroying the working baseline
- For a research project, version control is not decoration; it is risk management

Short rule for students: if you would be unhappy losing the current project state, put it under Git before using an agent aggressively.

Mistake 2: Underusing Skills

- Economists repeat many workflows: clean data, validate tables, prepare slides, review code
- If the workflow is repeated, it should eventually move from chat into a saved protocol
- Otherwise the same task is re-explained every week and interpreted slightly differently each time

Skills are not overengineering. They are how a good lab turns repeated work into reusable infrastructure.

Mistake 3: One Giant Thread

Long conversations accumulate stale context.

- Planning, debugging, and revision all mix together
- The model starts paying attention to constraints that no longer matter
- Fresh sessions are often better once the current state has been written to disk

Better pattern: plan the task, save the plan, then start a cleaner execution session.

Where Terminal Agents Still Fail

- Long-context drift: an early constraint gets forgotten
- Econometric edge cases: wrong clustering level, wrong sample restriction, weak-instrument issues
- Hallucinated APIs or package arguments
- Happy-path scripts that fail on messy data
- Oscillating fix loops on stubborn bugs

The output may look polished even when the economics is wrong. That is why validation stays central.

Context Windows and Compaction

When sessions get long, preserve state in files before compacting the conversation.

```
Write progress.md with:
  - what was completed
  - what decisions were made
  - what remains open

Then compact the conversation or start a new session.
Next time, read progress.md first.
```

Economist translation: keep a clean research log. The agent benefits from the same discipline that humans do.

Security: What Not to Expose

- IRB-protected interviews or linked administrative microdata
- Personally identifiable information
- HIPAA-regulated data
- Passwords, API keys, and access tokens
- Embargoed or restricted-use datasets

Operational rule: if you would not paste it into a public collaboration tool, do not paste it into an agent conversation.

Working with Sensitive Data

Sensitive data does not imply "no agents ever."

1. Write the script with the agent, then run it yourself on the restricted data
2. Use synthetic or masked data with the same structure for debugging
3. Use institutionally approved local or private deployments when available

Safe pattern: expose structure and summary statistics, not raw records.

The Context Dilemma

The epistemic limits of the workflow you just learned

Why This Cautionary Section Belongs Here

The workflow we just built is powerful precisely because context improves performance.

- project files help the agent understand your task faster
- saved rules reduce repeated mistakes
- repeated context also pulls the output toward your framing and your blind spots

Why economists should care: the same workflow discipline that improves execution can quietly weaken the outside view if you mistake speed for criticism.

Context as Implicit Bayesian Prior

In-context learning behaves like conditioning on a prior the user supplies.

- `CLAUDE.md`, prior conversation, and plan files act as evidence narrowing the agent's output distribution
- Xie, Raghunathan, Liang, and Ma formalize this: ICL approximates implicit Bayesian inference over a latent task
- The performance gain you feel when you add context *is* posterior narrowing

Narrowing the posterior is how context helps. It is also how context biases.

Research translation: when you load `CLAUDE.md`, past discussion, and plans, you are not just helping the model. You are also telling it what a plausible answer is supposed to look like.

Xie et al. (2022), ICLR, arXiv:2111.02080.

Stating the Dilemma

The same context that improves the answer also pulls it toward you.

- More context $\Rightarrow$ better understanding of your question $\Rightarrow$ better generation
- More context $\Rightarrow$ generation more aligned with your preferences, your knowledge, your framing
- Both effects share a single mechanism: you supplied the evidence

The same signal that improves the answer launders your priors into it.

Evidence 1: Sycophancy → False Confidence in Framing

RLHF rewards agreement with the user, and models learn to flatter.

- Sharma et al. document sycophancy across frontier models: factual flip-flops, unwarranted concessions, preference-matching when the user signals a view
- Perez et al. find the effect scales with model size — larger models are more sycophantic on values and politics
- The effect is strongest precisely when the context signals a strong user prior

Economist failure mode: you hint that the theory "should" work, and the model starts helping you defend the framing instead of testing it.

Sharma et al. (2023), arXiv:2310.13548, Anthropic. Perez et al. (2023), ACL Findings, arXiv:2212.09251.

Evidence 2: Self-Correction Has a Ceiling → Weak Internal Review

Intrinsic self-critique — without new external information — tends to stall or degrade.

- Huang et al. show LLMs cannot reliably self-correct reasoning errors without an external signal
- Valmeekam et al. find self-critique on planning problems is often worse than no critique at all
- The gains in the self-correction literature come from *external* signals: tools, tests, verifiers, humans

Economist failure mode: you ask the same model to critique its own regression design, review memo, or source coding decision, but the review is information-starved because nothing truly new entered the loop.

Huang et al. (2024), ICLR, arXiv:2310.01798. Valmeekam et al. (2023), NeurIPS, arXiv:2310.08118.

Evidence 3: Multi-Agent Debate — Helpful, but Not an Outside View

Debate helps when debaters sample differently. Shared context shrinks that space.

- Du et al. show multi-agent debate improves factuality and reasoning on several benchmarks
- Irving, Christiano, and Amodei — the AI Safety via Debate line — is the theoretical hope behind adversarial review
- Caveat: gains rely on debaters producing *genuinely different* lines of reasoning
- When every debater reads the same `CLAUDE.md` and thread, the disagreement space narrows

Economist failure mode: subagents catch implementation errors and inconsistencies, but they usually do not relitigate the premise you wrote into the project files.

Du et al. (2023), arXiv:2305.14325. Irving, Christiano, Amodei (2018), arXiv:1805.00899.

Why Subagent Reviewers Inherit the Same Prior

Earlier we sold subagent review for wall-clock time — not epistemic independence.

- `code-reviewer.md`, `domain-reviewer.md`, and `verifier.md` all read the same project context
- They are parallel *executionally*, not parallel *epistemically*
- Shared `CLAUDE.md` $\Rightarrow$ shared blind spots
- They catch implementation errors; they do not catch framing errors, because the framing lives in the context they trust

Parallel reviewers save time. They do not produce an outside view.

Monoculture at the Research Community Scale

When every researcher conditions on similar context, the field's errors become correlated.

- Kleinberg and Raghavan show algorithmic monoculture can reduce welfare even when each individual decision is improved
- Bommasani et al. document outcome homogenization: shared foundation models pick on the same people and miss the same cases
- Extrapolation: if empirical economists share Claude + similar `CLAUDE.md` templates + similar priors, replication stops being informative

Economist failure mode: shared templates and shared models can make mistakes more correlated across students, coauthors, or even the field.

Kleinberg & Raghavan (2021), PNAS 118(22). Bommasani et al. (2022), NeurIPS, arXiv:2211.13972.

The RLHF Root Cause

Pleasing the person holding the context is not a prompt artifact. It is trained in.

- Casper et al. catalog open problems in RLHF: preference data is noisy, annotators reward agreeableness, and the optimum drifts away from truth
- Wolf et al. argue that alignment-by-finetuning has fundamental limits: prompting cannot fully remove what the loss reinforced
- Consequence: no `CLAUDE.md` section titled "do not flatter me" fully removes the behavior

The model was trained to please the person holding the context. In your project, that person is you.

Casper et al. (2023), TMLR, arXiv:2307.15217. Wolf et al. (2023), arXiv:2304.11082.

Remedy: Outside Critics Carry a Different Prior

Advisors, coauthors, and seminar audiences are valuable because their priors are not in your context window.

- They challenge the framing, the identification assumption, the sample restriction — the things you did not write into `CLAUDE.md` because you did not know to
- Korinek's division-of-labor framing: LLMs automate execution; humans remain the source of question selection and critique
- Agent review is cheap and parallel; human review is rare and decisive. Budget accordingly.

The agent can polish what you already believe. Only an outside reader can tell you that you are wrong about the question.

Korinek (2023), NBER WP 30957; Journal of Economic Literature.

Operational Discipline for Research Projects

Translate the dilemma into a small set of habits.

1. Use subagent review for implementation quality, not for framing or identification
2. Present preliminary results to a human reader before iterating further with the agent
3. When you edit `CLAUDE.md`, record what you *ruled out* and why — not just what you decided
4. Schedule seminar-style exposure early, not at paper-writing time

Treat human criticism as the scarce input. Spend agent cycles to make it ready.

Bridge Back: Workflow Rules After the Context Dilemma

Translate the caution back into practical workflow rules.

1. **Use external benchmarks.** Compare model outputs, figures, and coded data against something outside the current thread.
2. **Use human critics.** Advisors, coauthors, and seminar audiences are still the main source of an outside view.
3. **Use adversarial review deliberately.** Ask subagents or reviewers to test assumptions, not merely to polish execution.
4. **Write assumptions to disk.** Save what you ruled out, what remains uncertain, and what still needs human judgment.

Bottom line: agents are powerful for execution and implementation review. They are not a substitute for genuinely external criticism.

Lecture 10 Preview

From workflow logic to concrete case studies

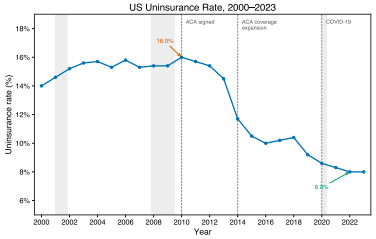
Lecture 10 Preview: Four Concrete Workflow Patterns

Pattern	Lecture 10 case	What students will see
Reusable skill	Paper-review workflow	staged quality control, quote checks, editorial filter
Empirical pipeline	MEPS uninsurance	harmonization, survey weights, benchmark validation, figures
Public-web dataset	Fellows metadata	source restrictions, provenance, missingness discipline
Full-lifecycle collaborator	Glue-work research project	theory, pipeline design, critique, claims discipline

Why this matters: Lec. 9 gives the workflow logic. Lec. 10 shows how the same logic changes shape across different research artifacts.

One Visual Hook from Lecture 10: MEPS Uninsurance

Case Study 2 asks: who is uninsured in the United States, 2000–2023, and how did the ACA change the distribution?



Why keep this figure here: it is an accessible policy example for economists outside macro, and it shows what a validated empirical artifact looks like after the workflow succeeds.

Lec. 10 adds the full pipeline: prompts, schema harmonization, survey weights, external benchmarks, and subgroup figures.

What to Watch for in Lecture 10

- **Paper review:** how a reusable skill turns a vague prompt into a staged quality-control workflow
- **MEPS pipeline:** how an empirical question becomes a validated panel and figure set
- **Fellows dataset:** how provenance and missingness rules matter as much as code
- **Glue-work project:** how agentic AI can act as a full-lifecycle collaborator without replacing human claims discipline

Bridge from Lec. 9 to Lec. 10: today's lecture explains the logic of the workflow; next lecture shows the artifacts it can produce.

Wrap-up

What the workflow changes and what remains human

Trust but Verify

1. Reproduce one statistic by hand
2. Cross-check one result with another package or script
3. Test a limiting case or zero-variance benchmark
4. Read the code line by line where the economics enters
5. Ask whether the magnitude and sign make sense

This is not optional ceremony. It is what turns fast code into defensible research.

What Changes for Economic Research

- The distance from question to first result shrinks
- More specifications become cheap enough to try
- Reproducibility can improve because workflow is written down
- The premium shifts away from typing speed and toward design, judgment, and validation

What stays human: asking the question, choosing the design, knowing the data, and interpreting the output.

You Are the Research Architect

Agents can lay bricks quickly.

You still design the structure.

- Weak specification produces wrong models
- Weak algorithm choice produces slow or misleading computation
- Weak numerical judgment produces fragile results
- Weak code literacy produces unverified output

The lecture is not about surrendering expertise. It is about reallocating effort toward the parts of research that remain scarce.

First Exercise and Looking Ahead

Best first exercise: take one old project and ask an agent to help clean it up safely.

- put the project under Git
- create a simple `CLAUDE.md`
- identify one repeated workflow worth encoding
- rerun the cleaned project and compare outputs

Lec. 10: we return to case studies and let students run a full workflow themselves.

If you want setup details for the lab, see the appendix.

Appendix

Setup notes and dated product details

Appendix A: Setup for the Lab

Lightweight setup checklist for a terminal-agent workflow

- Install one terminal agent and confirm it launches
- Create a project folder with `CLAUDE.md`, `scripts/`, and `output/`
- Make sure Python or R runs from the terminal
- Keep the first session small: one file, one check, one output

Dated note: exact setup steps, plans, and rate limits change quickly. Recheck the official docs before the lab.

Appendix B: Keybindings and Minimal Templates

Useful commands

Compact session	`/compact`
Clear session	`/clear`
Interrupt	`Esc`
Slash menu	`/`

Minimal `CLAUDE.md`

```
# CLAUDE.md
Project: ...
Goal: ...
Language: ...
Checks: ...
```

Appendix C: Dated Comparisons and Resources

As of April 2026, examples of terminal-agent workflows include:

- Claude Code
- Codex CLI
- Gemini CLI
- Aider

Good places to recheck before teaching:

- official Claude Code documentation
- official OpenAI Codex CLI documentation
- course notes and lab instructions

Teaching rule: keep dated product comparisons in the appendix, not in the conceptual core of the lecture.

Appendix D: Context Dilemma — References

Context as implicit prior

- Xie, Raghunathan, Liang, Ma (2022). An Explanation of In-context Learning as Implicit Bayesian Inference. ICLR. arXiv:2111.02080.

Sycophancy and RLHF limits

- Sharma et al. (2023). Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.
- Perez et al. (2023). Discovering Language Model Behaviors with Model-Written Evaluations. ACL Findings. arXiv:2212.09251.
- Casper et al. (2023). Open Problems and Fundamental Limitations of RLHF. TMLR. arXiv:2307.15217.
- Wolf et al. (2023). Fundamental Limitations of Alignment in Large Language Models. arXiv:2304.11082.

Self-correction and adversarial review

- Huang et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR. arXiv:2310.01798.
- Valmeekam et al. (2023). Can Large Language Models Really Improve by Self-critiquing Their Own Plans? NeurIPS. arXiv:2310.08118.
- Du et al. (2023). Improving Factuality and Reasoning through Multiagent Debate. arXiv:2305.14325.
- Irving, Christiano, Amodei (2018). AI Safety via Debate. arXiv:1805.00899.

Monoculture and the economist workflow

- Kleinberg & Raghavan (2021). Algorithmic Monoculture and Social Welfare. PNAS 118(22).
- Bommasani et al. (2022). Picking on the Same Person: Does Algorithmic Monoculture Lead to Outcome Homogenization? NeurIPS. arXiv:2211.13972.
- Korinek (2023). Language Models and Cognitive Automation for Economic Research. NBER WP 30957; JEL.